

INTERNSHIP REPORT

SUBMITTED BY:

- | | |
|--------------------------|-----------------------|
| 1. NANDALA SAIKIRAN | [ROLL NO. 22011P0412] |
| 2. VAINALA SUDHEER | [ROLL NO. 22011P0417] |
| 3. MAILA GYANENDRA SIMHA | [ROLL NO. 22011P0437] |
| 4. NADENDLA JESU WESLY | [ROLL NO. 22011P0450] |

From Department of **ELECTRONICS AND COMMUNICATION** at **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KUKATPALLY, HYDERABAD.**

The Internship was completed under the Guidance of **Mr. BS RAO [DGM, SED].**

DURATION OF THE INTERNSHIP: **12-05-2025 TO 09-06-2025**

Signature of Guide
B. Srinivas RAO
उप महा प्रबंधक Deputy General Manager
ई सी आई एल, हैदराबाद
SED, ECIL, HYDERABAD-500 062



ELECTRONICS CORPORATION OF INDIA LIMITED

A Govt. of India (Dept. of Atomic Energy) Enterprise

IMPLEMENTATION OF SHA-512 ALGORITHM ON AMD Zynq™ 7000 SoC ZC702 Evaluation Kit

A fully pipelined SHA-512 core was designed and implemented on the **AMD Zynq™ 7000 SoC ZC702 Evaluation Kit**, based on the NIST FIPS 180-4 specification. SHA-512 is a more robust cryptographic hash function compared to SHA-256, operating with a larger internal state and producing a 512-bit hash output, which significantly enhances its security against brute-force and collision attacks.

The algorithm was divided into modular stages—message scheduling, compression, and state update—to optimize utilization of the ZC702's programmable logic resources (LUTs, DSP slices, and block RAM). Unlike SHA-256's 512-bit blocks and 32-bit words, the SHA-512 message scheduler expands each **1024-bit block into 80 64-bit words** using large-sigma functions and feeds them sequentially into the compression core. The compression unit performs **80 rounds of transformation** using big-sigma, choice, and majority operations, iteratively updating the **eight 64-bit internal hash state variables**.

All functional blocks were built using low-level hardware logic, including shifts, rotations, and bitwise operations, enabling efficient synthesis and high performance on the Zynq-7000's architecture. A top-level wrapper managed data I/O, pipeline registers, and state transitions, ensuring a continuous flow of data through the design. Vivado's synthesis and implementation tools were used to achieve timing closure and optimize resource mapping across the FPGA fabric. The design aimed for a high operating frequency and was verified to produce correct 512-bit hash outputs using standard NIST test vectors.

To ensure functional correctness under real operating conditions, on-chip verification was performed using Vivado's Integrated Logic Analyzer (ILA). The ILA enabled internal signal capture and live debugging, making it possible to inspect and validate the datapath in real time. This not only confirmed the design's correctness but also helped fine-tune timing and control logic during development. The final implementation demonstrates a robust and resource-efficient hardware realization of SHA-512, delivering strong throughput and accuracy. Its modular and pipelined structure provides a solid foundation for future cryptographic extensions or integration into larger secure systems.

Project Description

1. Padding bits

The first step is to carry out the padding function in which we append a certain number of bits to the plaintext message to increase its length, which should be exactly 128 bits less than an exact multiple of 1024.

When we are appending these bits at the end of the message we start with a '1' and then keep on adding '0' till the moment we reach the last bit that needs to be appended under padding and leave the 128 bits after that.



2. Length bits

Now, we add the remaining 128 bits left to this entire block to make it an exact multiple of 1024, so that the whole thing can be broken down in 'n' number of 1024 blocks of message that we will apply our operation on. The way to calculate the rest of the 128 bits is by calculating the modulo with 2^{64} .

Once done, we append it to the padded bit and the original message to make the entire length of the block to be "n x 1024" in length.

3. Initialize the buffers

Now, that we have "n x 1024" length bit message that we need to hash, let us focus on the parts of the hashing function itself. To carry on the hash and the computations required, we need to have some default values initialized.

```
a = 0x6a09e667f3bcc908
b = 0xbb67ae8584caa73b
c = 0x3c6ef372fe94f82b
d = 0xa54ff53a5f1d36f1
e = 0x510e527fade682d1
f = 0x9b05688c2b3e6c1f
g = 0x1f83d9abfb41bd6b
h = 0x5be0cd19137e2179
```

These are the values of the buffer that we will need, there are other default values that we need to initialize as well.

These are the value for the 'k' variable which we will be using.

```

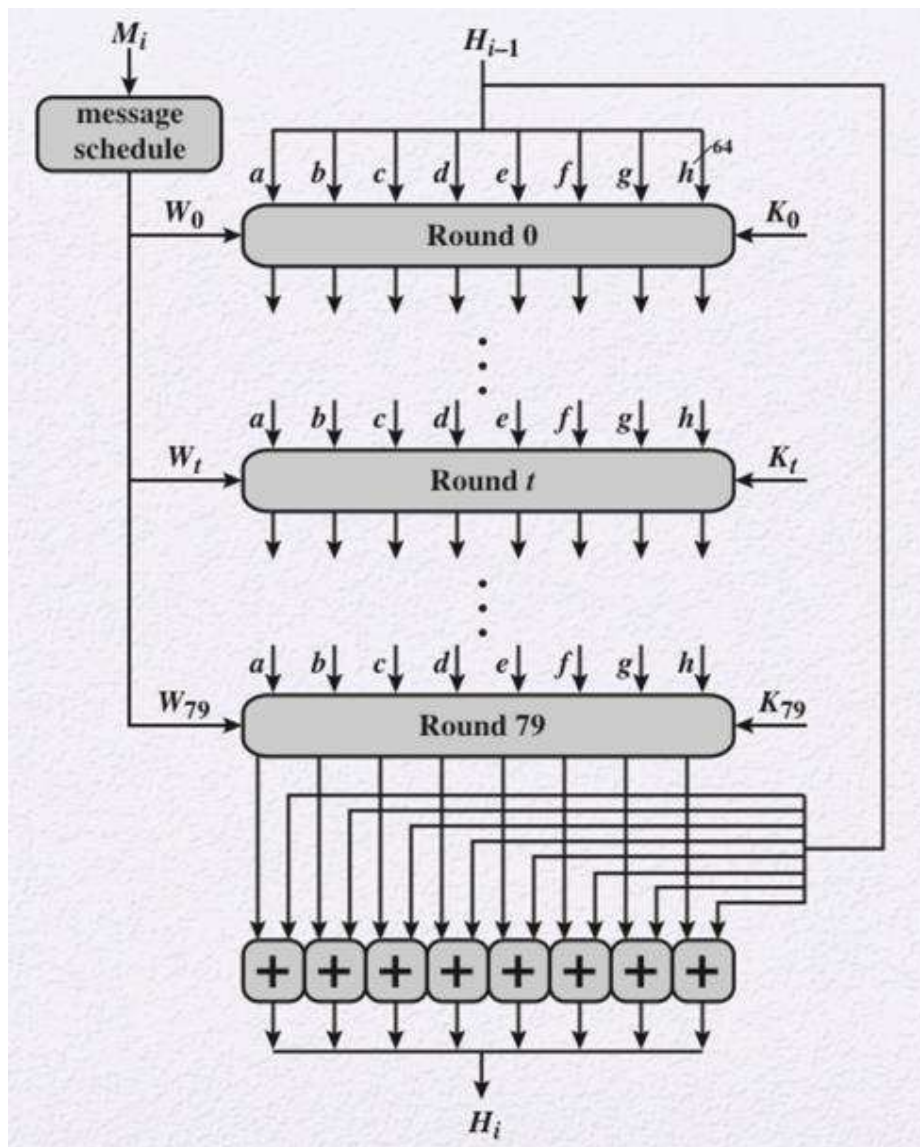
k[0..79] := [ 0x428a2f98d728ae22, 0x7137449123ef65cd, 0xb5c0fbcfec4d3b2f, 0xe9b5dba58189dbbc, 0x3956c25bf348b538,
  0x59f111f1b605d019, 0x923f82a4af194f9b, 0xab1c5ed5da6d8118, 0xd807aa98a3030242, 0x12835b0145706fbe,
  0x243185be4ee4b28c, 0x550c7dc3d5ffb4e2, 0x72be5d74f27b896f, 0x80deb1fe3b1696b1, 0x9bdc06a725c71235,
  0xc19bf174cf692694, 0xe49b69c19ef14ad2, 0xefbe4786384f25e3, 0x0fc19dc68b8cd5b5, 0x240ca1cc77ac9c65,
  0x2de92cc6f592b0275, 0x4a7484aa6eae483, 0x5cb0a9dcbd41fbd4, 0x76f988da831153b5, 0x983e5152ee66dfab,
  0xa831c66d2db43210, 0xb00327c898fb213f, 0xbf597fc7beef0ee4, 0xc6e00bf33da88fc2, 0xd5a79147930aa725,
  0x06ca6351e003826f, 0x142929670a0e6e70, 0x27b70a8546d22ffc, 0x2e1b21385c26c926, 0x4d2c6dfc5ac42aed,
  0x53380d139d95b3df, 0x650a73548baf63de, 0x766a0abb3c77b2a8, 0x81c2c92e47edae66, 0x92722c851482353b,
  0xa2bfe8a14cf10364, 0xa81a664bbc423001, 0xc24b8b70d0f89791, 0xc76c51a30654be30, 0xd192e819d6ef5218,
  0xd69906245565a910, 0xf40e35855771202a, 0x106aa07032bdbl8, 0x19a4c116b8d2d0c8, 0x1e376c085141ab53,
  0x2748774cdf8eeb99, 0x34b0bcb5e19b48a8, 0x391c0cb3c5c95a63, 0x4ed8aa4ae3418acb, 0x5b9cca4f7763e373,
  0x682e6ff3d6b2b8a3, 0x748f82ee5defb2fc, 0x78a5636f43172f60, 0x84c87814a1f0ab72, 0x8cc702081a6439ec,
  0x90befffa23631e28, 0xa4506cebd82bde9, 0xbef9a3f7b2c67915, 0xc67178f2e372532b, 0xca273ecee626619c,
  0xd186b8c721c0c207, 0xeda7dd6cde0eb1e, 0xf57d4f7fee6ed178, 0xf06f067aa72176fba, 0xa637dc5a2c898a6,
  0x113f9804bef90dae, 0x1b710b35131c471b, 0x28db77f523047d84, 0x32caab7b40c72493, 0x3c9ebe0a15c9bebc,
  0x431d67c49c100d4c, 0x4cc5d4becb3e42b6, 0x597f299cfc657e2a, 0x5fcb6fab3ad6faec, 0x6c44198c4a475817]

```

The reason for initiating these values will be clear to you in the very next step that we will explore.

4. Compression function

Now, we need to have a wider look at the hash function so that we can understand what is happening. First of all, we take 1024 bits of messages and divide the complete message in 'n' bits.



Having a look at the above image you get the idea that we will be working on 80 rounds and the input, W will be derived from the 1024 bits, which is further divided into 16 parts. The value of W from 0 to 15 is the message in plaintext but the values of W from 16 to 79 is calculated from the previous 16 blocks and we have the formula mentioned below.

$$W(t) = \sigma^1(W^{t-2}) + W^{t-7} + \sigma^0(W^{t-15}) + W^{t-16}$$

where,

$$\sigma^0(x) = \text{ROTR}^1(x) \oplus \text{ROTR}^8(x) \oplus \text{SHR}^7(x)$$

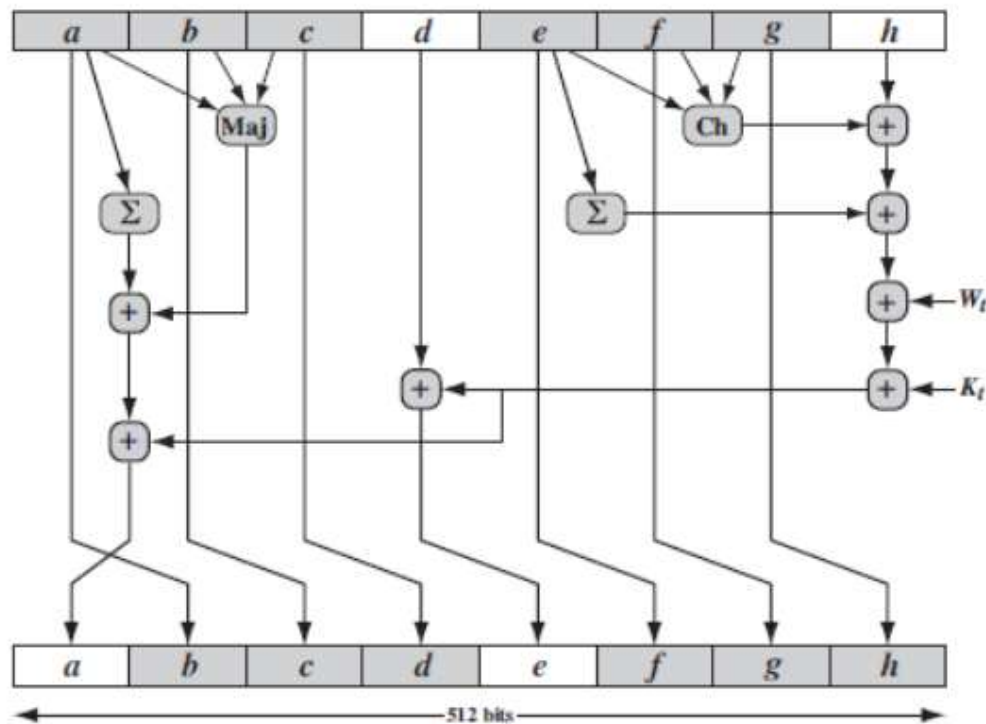
$$\sigma^1(x) = \text{ROTR}^{19}(x) \oplus \text{ROTR}^{61}(x) \oplus \text{SHR}^6(x)$$

$\text{ROTR}^n(x)$ = Circular right rotation of 'x' by 'n' bits

$\text{SHR}^n(x)$ = Circular right shift of 'x' by 'n' bits

$\oplus$ = addition modulo 2^{64}

Now that we know the methodology to obtain the values of W for 0 to 79 and already have the values for K as well for all the rounds from 0 to 79 we can proceed ahead and see where and how we do put in these values for the computation of the hash.



Depiction of a “round”

In the image above we can see exactly what happens in each round and now that we have the values and formulas for each of the functions carried out we can perform the entire hashing process.

$$\text{Ch}(E, F, G) = (E \text{ AND } F) \text{ XOR } ((\text{NOT } E) \text{ AND } G)$$

$$\text{Ma}(A, B, C) = (A \text{ AND } B) \text{ XOR } (A \text{ AND } C) \text{ XOR } (B \text{ AND } C)$$

$$\Sigma(A) = (A \ggg 2) \text{ XOR } (A \ggg 13) \text{ XOR } (A \ggg 22)$$

$$\Sigma(E) = (E \ggg 6) \text{ XOR } (E \ggg 11) \text{ XOR } (E \ggg 25)$$

$$+ = \text{addition modulo } 2^{64}$$

These are the functions that are performed in each of the 80 rounds that are performed over and over for 'n' number of times.

5. Output

The output from every round acts as an input for the next round and this process keeps on continuing till the last bit of the message go through one of the rounds and the output from the last block of the last message is the hash code. The length of the output is 512 bits.

Conclusion

The SHA-512 hashing algorithm is currently one of the best and secured hashing algorithms after hashes like MD5 and SHA-1 has been broken down. Due to their complicated nature it is not well accepted and SHA-256 is a general standard, but the industry is slowly moving towards this hashing algorithm.

Let's have a look at the entire working of the SHA-512 hashing algorithm and see in a concise manner how it works.

The message that needs to be hashed, we first append a few bits to it, making it exactly 128 bits less than the multiple of 1024. The remaining 128 bits are added after we calculate the modulo of the original message with 2^{64} . Once we do that then we divide them in 'n' pieces of 1024 bits. After this we pass the 1024 bit blocks one by one into the compression function i.e. the set of 80 rounds of operations. Here we further divide them into 16 parts each of 64 bits. These 16 parts of 64 bits acts as inputs in each rounds of the operation in a sequential manner and W for 16 to 79 is calculated using the first 16 parts. We already have the default values of 'K' for each of the rounds. Now we begin with the rounds and the set of operation that needs to be performed over and over in each of these rounds. The output for one round is taken as the input for the next round and this process keeps on repeating till the 80th round of the last message block, takes place. The output obtained after the last round of the last message block is the hash code of the entire message.

Software Simulation of SHA 512 in Vivado Software

Verilog HDL Code

```
`timescale 1ns / 1ps

module sha512final (
    input wire clk,
    input wire reset,
    input wire [1023:0] message, // 1024-bit input block
    output reg [511:0] hash      // 512-bit output hash
);

// 64-bit rotate right function
function [63:0] ROR(input [63:0] x, input [10:0] n);
    ROR = (x >> n) | (x << (64 - n));
endfunction

// Initial hash values (SHA-512 constants)
reg [63:0] h0 = 64'h6a09e667f3bcc908;
reg [63:0] h1 = 64'hbb67ae8584caa73b;
reg [63:0] h2 = 64'h3c6ef372fe94f82b;
reg [63:0] h3 = 64'ha54ff53a5f1d36f1;
reg [63:0] h4 = 64'h510e527fade682d1;
reg [63:0] h5 = 64'h9b05688c2b3e6c1f;
reg [63:0] h6 = 64'h1f83d9abfb41bd6b;
reg [63:0] h7 = 64'h5be0cd19137e2179;

// SHA-512 round constants
reg [63:0] k [0:79];
initial begin
    k[0] = 64'h428a2f98d728ae22; k[1] = 64'h7137449123ef65cd;
    k[2] = 64'hb5c0fbcfec4d3b2f; k[3] = 64'he9b5dba58189dbbc;
    k[4] = 64'h3956c25bf348b538; k[5] = 64'h59f111f1b605d019;
    k[6] = 64'h923f82a4af194f9b; k[7] = 64'hab1c5ed5da6d8118;
    k[8] = 64'hd807aa98a3030242; k[9] = 64'h12835b0145706fbc;
    k[10] = 64'h243185be4ee4b28c; k[11] = 64'h550c7dc3d5ffb4e2;
    k[12] = 64'h72be5d74f27b896f; k[13] = 64'h80deb1fe3b1696b1;
    k[14] = 64'h9bdc06a725c71235; k[15] = 64'hc19bf174cf692694;
    k[16] = 64'he49b69c19ef14ad2; k[17] = 64'hefb4786384f25e3;
    k[18] = 64'h0fc19dc68b8cd5b5; k[19] = 64'h240ca1cc77ac9c65;
    k[20] = 64'h2de92c6f592b0275; k[21] = 64'h4a7484aa6ea6e483;
    k[22] = 64'h5cb0a9dcdbd41fbd4; k[23] = 64'h76f988da831153b5;
    k[24] = 64'h983e5152ee66dfab; k[25] = 64'ha831c66d2db43210;
    k[26] = 64'hb00327c898fb213f; k[27] = 64'hbf597fc7beef0ee4;
    k[28] = 64'hc6e00bf33da88fc2; k[29] = 64'hd5a79147930aa725;
    k[30] = 64'h06ca6351e003826f; k[31] = 64'h142929670a0e6e70;
    k[32] = 64'h27b70a8546d22ffc; k[33] = 64'h2e1b21385c26c926;
    k[34] = 64'h4d2c6dfc5ac42aed; k[35] = 64'h53380d139d95b3df;
    k[36] = 64'h650a73548baf63de; k[37] = 64'h766a0abb3c77b2a8;
    k[38] = 64'h81c2c92e47edaee6; k[39] = 64'h92722c851482353b;
    k[40] = 64'ha2bfe8a14cf10364; k[41] = 64'ha81a664bbc423001;
    k[42] = 64'hc24b8b70d0f89791; k[43] = 64'hc76c51a30654be30;
    k[44] = 64'hd192e819d6ef5218; k[45] = 64'hd69906245565a910;
    k[46] = 64'hf40e35855771202a; k[47] = 64'h106aa07032bbd1b8;
    k[48] = 64'h19a4c116b8d2d0c8; k[49] = 64'h1e376c085141ab53;
    k[50] = 64'h2748774cdf8eeb99; k[51] = 64'h34b0bcb5e19b48a8;
    k[52] = 64'h391c0cb3c5c95a63; k[53] = 64'h4ed8aa4ae3418acb;
    k[54] = 64'h5b9cca4f7763e373; k[55] = 64'h682e6ff3d6b2b8a3;
    k[56] = 64'h748f82ee5defb2fc; k[57] = 64'h78a5636f43172f60;
    k[58] = 64'h84c87814a1f0ab72; k[59] = 64'h8cc702081a6439ec;
    k[60] = 64'h90befffa23631e28; k[61] = 64'ha4506cebde82bde9;
    k[62] = 64'hbef9a3f7b2c67915; k[63] = 64'hc67178f2e372532b;
```

```

k[64] = 64'hca273ecee26619c; k[65] = 64'hd186b8c721c0c207;
k[66] = 64'heada7dd6cde0eb1e; k[67] = 64'hf57d4f7fee6ed178;
k[68] = 64'h06f067aa72176fba; k[69] = 64'h0a637dc5a2c898a6;
k[70] = 64'h113f9804bef90dae; k[71] = 64'h1b710b35131c471b;
k[72] = 64'h28db77f523047d84; k[73] = 64'h32caab7b40c72493;
k[74] = 64'h3c9ebe0a15c9bebc; k[75] = 64'h431d67c49c100d4c;
k[76] = 64'h4cc5d4becb3e42b6; k[77] = 64'h597f299cfc657e2a;
k[78] = 64'h5fcb6fab3ad6faec; k[79] = 64'h6c44198c4a475817;
end

// Working variables
reg [63:0] a, b, c, d, e, f, g, h;
reg [63:0] w [0:79];
reg [63:0] temp1, temp2;
integer i;

always @(posedge clk or posedge reset) begin
    if(reset) begin
        hash <= 512'b0;
        for (i = 0; i < 80; i = i + 1)
            w[i] <= 0;
        a = 0; b = 0; c = 0; d = 0;
        e = 0; f = 0; g = 0; h = 0;
    end else begin
        for (i = 0; i < 16; i = i + 1)
            w[i] = message[1023 - (i * 64) -: 64];

        for (i = 16; i < 80; i = i + 1)
            w[i] = (ROR(w[i-2], 19) ^ ROR(w[i-2], 61) ^ (w[i-2] >> 6)) + w[i-7] +
                (ROR(w[i-15], 1) ^ ROR(w[i-15], 8) ^ (w[i-15] >> 7)) + w[i-16];

        a = h0; b = h1; c = h2; d = h3;
        e = h4; f = h5; g = h6; h = h7;

        for (i = 0; i < 80; i = i + 1) begin
            temp1 = h + (ROR(e,14)^ROR(e,18)^ROR(e,41)) + ((e&f)^(~e&g)) + k[i] + w[i];
            temp2 = (ROR(a,28)^ROR(a,34)^ROR(a,39)) + ((a&b)^(a&c)^(b&c));
            h = g; g = f; f = e; e = d + temp1;
            d = c; c = b; b = a; a = temp1 + temp2;
        end

        h0 = h0 + a; h1 = h1 + b; h2 = h2 + c; h3 = h3 + d;
        h4 = h4 + e; h5 = h5 + f; h6 = h6 + g; h7 = h7 + h;

        hash = {h0, h1, h2, h3, h4, h5, h6, h7};
    end
end

endmodule

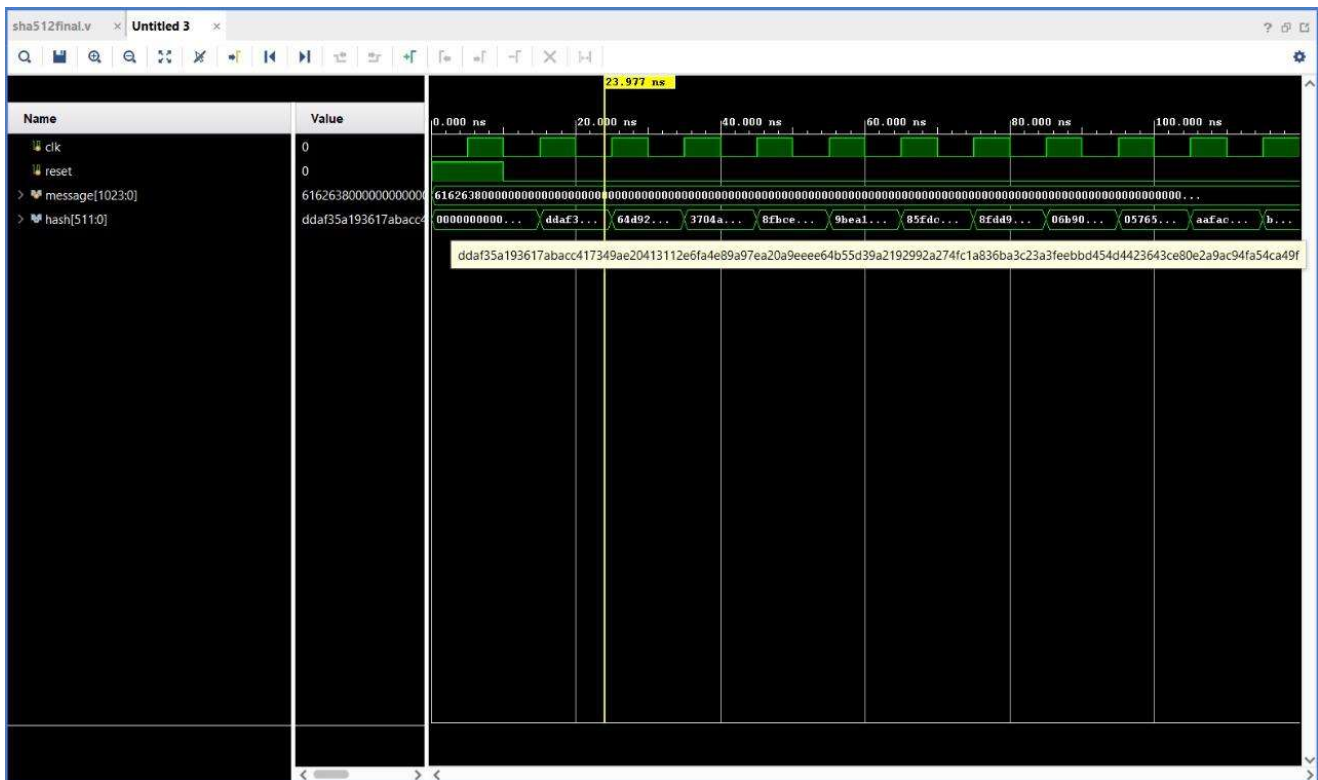
module sha512_tb;

    reg clk;
    reg reset;
    reg [1023:0] message;
    wire [511:0] hash;

    // Instantiate SHA-256 module
    sha512final uut (
        .clk(clk),
        .reset(reset),
        .message(message),
        .hash(hash)
    );

    // Clock generation
    always #5 clk = ~clk;

```

[illegible]

REFERENCES:

[Breaking Down : SHA-512 Algorithm | by Aditya Anand | InfoSec Write-ups](#)

<https://infosecwriteups.com/breaking-down-sha-512-algorithm-1fdb9cc9413a> - Aditya Anand

<https://bitcoinwiki.org/wiki/sha-512>